

SmartCow Tracker

Sistema de Rastreo y Monitoreo de Ganado Bovino

ARCHITECTURE DECISION RECORDS (ADR)

Formato MADR (Markdown Architectural Decision Records) · ISO/IEC 42010:2022

Total ADRs	10 decisiones documentadas
Versión	1.0.0
Fecha	Mayo 2026
Estado	Vigente

Introducción

¿Qué son los ADR?

Un Architecture Decision Record (ADR) es un documento de trabajo corto que captura una decisión arquitectónica importante tomada junto con su contexto y consecuencias. El concepto fue introducido por Michael Nygard y adoptado como práctica estándar de la industria bajo el formato MADR (Markdown Architectural Decision Records).

Este conjunto de ADRs ha sido elaborado conforme a los principios de descripción arquitectónica de IEEE/ISO/IEC 42010:2022, que exige documentar las decisiones de diseño con su justificación, alternativas consideradas y consecuencias sobre los atributos de calidad del sistema.

Índice de Decisiones

ID	Título de la Decisión	Estado	Área
ADR-001	Arquitectura monolítica modular en lugar de microservicios	ACEPTADA	Arquitectura general
ADR-002	PostgreSQL como base de datos relacional principal	ACEPTADA	Persistencia de datos
ADR-003	Redis para estado en tiempo real y pub/sub	ACEPTADA	Tiempo real / Cache
ADR-004	MQTT sobre HTTP para comunicación con dispositivos IoT	ACEPTADA	Comunicación IoT
ADR-005	React SPA con Vite en lugar de Next.js SSR	ACEPTADA	Frontend
ADR-006	Servicio de IA como contenedor Python independiente	ACEPTADA	Inteligencia Artificial
ADR-007	JWT RS256 con refresh token en cookie HttpOnly	ACEPTADA	Autenticación
ADR-008	Prisma ORM sobre consultas SQL directas	ACEPTADA	Acceso a datos
ADR-009	Bull MQ para procesamiento asíncrono de trabajos	ACEPTADA	Procesamiento background
ADR-010	Leaflet.js con OpenStreetMap sobre Google Maps	ACEPTADA	Mapas / Geolocalización

ADR-001 — Arquitectura monolítica modular en lugar de microservicios

Estado	ACEPTADA
Fecha	15 de enero de 2026
Decisores	Tech Lead, Arquitecto de Software
Consultados	Equipo de Desarrollo (2 devs)
Informados	Product Owner, Stakeholders

| Contexto y Problema

SmartCow Tracker debe gestionar múltiples dominios de negocio: autenticación, rastreo GPS, inventario de ganado, alertas, geocercas, analítica e IA. El equipo de desarrollo inicial es de 1-3 personas con experiencia en desarrollo full-stack pero sin experiencia operacional profunda en infraestructura de microservicios (Kubernetes, service mesh, distributed tracing, etc.). Se necesita una decisión sobre la estrategia de descomposición del sistema backend.

| Decisión

Se adopta una arquitectura de monolito modular (también conocida como modular monolith o majestic monolith) con separación estricta de dominios internos mediante interfaces bien definidas entre módulos. El sistema se despliega como un único proceso Node.js por entorno, pero con separación clara de responsabilidades que permita eventualmente extraer módulos como microservicios independientes si el crecimiento lo justifica.

| Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
Monolito Modular (Node.js)	Un solo proceso con módulos internos bien separados. Despliegue simple, depuración directa, sin overhead de red entre módulos.	✓ ELEGIDA
Microservicios (Docker + K8s)	Servicios independientes por dominio. Escalado granular pero requiere Kubernetes, API gateway, service discovery, distributed tracing y experiencia operacional avanzada.	✗ Descartada
Serverless Functions	AWS Lambda o similar. Sin servidor que gestionar, pero cold starts inaceptables para WebSockets y procesamiento de telemetría continuo.	✗ Descartada
Monolito Clásico (sin separación)	Sin módulos internos definidos. Simple de iniciar pero genera deuda técnica masiva a mediano plazo. Descartado por falta de mantenibilidad.	✗ Descartada

| Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
Despliegue y operación simples con 1-3 desarrolladores.	Mayor acoplamiento de despliegue: un bug crítico afecta todo.
Depuración directa sin tracing distribuido.	Escalado horizontal solo del proceso completo, no por módulo.
Latencia interna cero entre módulos (llamadas de función).	A largo plazo, sin disciplina, puede degradarse a bola de barro.
Menor costo de infraestructura inicial (un solo contenedor).	Limitación de stack tecnológico: todos los módulos en Node.js.
Refactoring a microservicios posible cuando se justifique.	
Pruebas de integración más sencillas y rápidas.	

Consecuencias

- Se establece como regla: ningún módulo puede importar directamente los internos de otro módulo. Toda comunicación entre módulos es a través de interfaces exportadas públicamente.
- Se documenta un 'mapa de módulos' que define las fronteras y contratos entre dominios para facilitar una futura extracción si el sistema escala a >10,000 dispositivos.
- El Job Worker se despliega como proceso separado (diferente endpoint) pero comparte el mismo código base, actuando como primer paso hacia la separación.
- La decisión se revisará si el sistema supera los 5,000 dispositivos IoT activos simultáneos o si el equipo crece a más de 8 desarrolladores.

Referencias

- Sam Newman — Monolith to Microservices (2019). O'Reilly Media.
- Martin Fowler — MonolithFirst (<https://martinfowler.com/bliki/MonolithFirst.html>).

ADR-002 — PostgreSQL como base de datos relacional principal	
Estado	ACEPTADA
Fecha	15 de enero de 2026
Decisores	Tech Lead, Arquitecto de Software
Consultados	Equipo de Desarrollo
Informados	Product Owner

Contexto y Problema

El sistema necesita almacenar datos con características muy diversas: datos estructurados relacionales (fichas de ganado, usuarios, fincas), series de tiempo de alta frecuencia (posiciones GPS cada 10 segundos por animal), datos geoespaciales (polígonos de geocercas), datos semiestructurados (payload flexible de sensores IoT) y relaciones complejas entre entidades. Se necesita elegir el motor de base de datos principal que satisfaga todos estos requisitos con un solo sistema.

Decisión

Se adopta PostgreSQL 15 como única base de datos relacional del sistema, aprovechando sus extensiones nativas para cubrir todos los tipos de datos requeridos: PostGIS para datos geoespaciales, JSONB para datos semiestructurados, tipos de array nativos y particionamiento nativo por rango de fechas para las tablas de series de tiempo.

Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
PostgreSQL 15 + PostGIS	BD relacional con soporte nativo geoespacial, JSONB, particionamiento y ACID completo. Cubre todos los casos de uso con un solo motor.	✓ ELEGIDA
MongoDB (NoSQL)	Flexible para esquemas variables, pero sin transacciones ACID completas entre colecciones, sin soporte geoespacial avanzado nativo y menor madurez en operaciones.	✗ Descartada
PostgreSQL + InfluxDB	PostgreSQL para datos relacionales e InfluxDB para series de tiempo GPS. Más rendimiento en series de tiempo pero duplica la complejidad operacional (dos BD que mantener).	✗ Descartada
MySQL 8	Alternativa relacional madura pero sin PostGIS nativo, sin JSONB real (solo JSON), menor soporte para operaciones geoespaciales complejas.	✗ Descartada

Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
PostGIS: operaciones geoespaciales nativas (ST_Contains, ST_Distance).	Escrituras de alta frecuencia (1,500 registros/min) requieren tuning de índices.
JSONB: almacenamiento e indexación eficiente de payloads IoT variables.	Escalado vertical más costoso que soluciones NoSQL para volúmenes extremos.
Particionamiento nativo por rango de fechas para tablas de alta frecuencia.	Particionamiento y archivado requieren configuración inicial adicional.
Transacciones ACID para garantizar consistencia en operaciones críticas.	
Prisma ORM con soporte completo y tipos generados automáticamente.	
Un solo sistema que aprender, operar y hacer backup.	
Madurez y ecosistema masivo. Disponible en todos los proveedores cloud.	

Consecuencias

- Las tablas GpsHistory y VitalHistory usarán particionamiento por rango mensual desde el inicio para garantizar rendimiento a largo plazo.
- Las posiciones GPS más recientes (dato de lectura más frecuente) NO se leen de PostgreSQL sino de Redis, eliminando el problema de escrituras concurrentes.
- Se instala la extensión PostGIS en el esquema de producción para futura implementación de consultas geoespaciales avanzadas (ej: 'todos los animales en un radio de 500m de un punto').
- Conexión gestionada con PgBouncer en modo transaction pooling para evitar agotamiento del pool de conexiones bajo carga.

Referencias

- PostGIS Documentation — <https://postgis.net/documentation/>
- PostgreSQL 15 Release Notes — Table Partitioning improvements.

ADR-003 — Redis para estado en tiempo real y pub/sub WebSocket	
Estado	ACEPTADA
Fecha	18 de enero de 2026
Decisores	Tech Lead
Consultados	Equipo de Desarrollo
Informados	Product Owner, DevOps

Contexto y Problema

El sistema debe mostrar las posiciones GPS de hasta 247 animales actualizándose cada 10 segundos en los navegadores de múltiples usuarios simultáneos. Leer la posición más reciente de cada animal directamente de PostgreSQL en cada actualización implicaría ~25 queries/segundo solo para posiciones, creando un cuello de botella inaceptable. Además, si el backend escala a múltiples instancias, los WebSockets de diferentes clientes conectados a diferentes instancias no podrían comunicarse entre sí sin un mecanismo de mensajería compartida.

Decisión

Se adopta Redis 7 como capa de estado en tiempo real con dos responsabilidades principales: (1) almacenar la posición GPS más reciente de cada animal en estructuras HASH de lectura ultrarrápida (< 2ms), y (2) implementar el patrón pub/sub para distribuir eventos de tiempo real entre múltiples instancias del API Server, permitiendo que los WebSocket Gateways de diferentes instancias emitan eventos a sus clientes conectados.

Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
Redis 7 (HASH + pub/sub)	Cache de posiciones en estructuras HASH (O(1) lectura/escritura). Pub/sub nativo para distribuir eventos WebSocket entre instancias. Ampliamente soportado.	✓ ELEGIDA
Polling a PostgreSQL	Sin Redis adicional. Clientes hacen polling HTTP cada 10s. Genera 25 req/s solo por posiciones. Latencia alta. No escala con número de usuarios.	✗ Descartada
Apache Kafka	Ideal para event streaming de alto volumen, pero sobredimensionado para este caso. Requiere ZooKeeper/KRaft, complejo de operar y configurar para un equipo pequeño.	✗ Descartada
Memoria del proceso Node.js	Sin dependencia externa, pero el estado se pierde al reiniciar el proceso y es imposible compartirlo entre múltiples instancias.	✗ Descartada

Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
Lectura de posición actual en < 2ms (HGET es O(1)).	Dependencia adicional de infraestructura que operar y monitorear.
Permite escalado horizontal del API Server sin perder estado.	Datos en Redis son efímeros por naturaleza: requiere sincronización con PostgreSQL.
Pub/sub nativo elimina dependencia de una librería de mensajería externa.	Coste adicional de hosting (Upstash Redis free tier o \$0.2/100k cmds).
TTL automático para expirar sesiones y tokens de challenge.	Complejidad de consistencia eventual entre Redis y PostgreSQL.
Bull MQ usa Redis como backend: una sola dependencia para dos propósitos.	
Persistencia AOF configurable para recuperar estado tras reinicio.	

Consecuencias

- Redis NO es la fuente de verdad. PostgreSQL sigue siendo el sistema de registro. Redis es solo una capa de aceleración.
- Ante reinicio de Redis, el Worker repuebla las posiciones actuales desde el último registro de PostgreSQL automáticamente (startup recovery job).
- Se usa Upstash Redis en producción para simplicidad operacional (serverless Redis, sin servidor que gestionar).
- La estructura de claves en Redis sigue la convención: animal:{animalId}:position, farm:{farmId}:channel, session:{token}:user.

ADR-004 — MQTT sobre HTTP para comunicación con dispositivos IoT	
Estado	ACEPTADA
Fecha	20 de enero de 2026
Decisores	Tech Lead, Arquitecto IoT
Consultados	Equipo de Hardware, Equipo de Backend
Informados	Product Owner

Contexto y Problema

Cada collar IoT (ESP32 + GPS + sensores) debe transmitir datos de telemetría al servidor cada 10 segundos desde entornos rurales con conectividad limitada (LoRa o GSM 2G/3G). El protocolo de comunicación debe ser: eficiente en consumo de batería del dispositivo, tolerante a pérdida de conexión intermitente, ligero en overhead de red (los dispositivos tienen ancho de banda restringido), y capaz de garantizar la entrega de mensajes críticos.

Decisión

Se adopta el protocolo MQTT 3.1.1 sobre TLS (MQTTS en el puerto 8883) como protocolo de comunicación entre los collares IoT y el servidor backend. Se despliega Mosquitto 2.x como broker MQTT. El backend (API Server) se suscribe a los topics relevantes via el cliente MQTT.js. HTTP REST queda como protocolo de fallback para configuración inicial de dispositivos y actualizaciones de firmware.

Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
MQTT 3.1.1 + Mosquitto	Diseñado específicamente para IoT. Overhead de protocolo mínimo (2 bytes header). QoS configurable. Keep-alive integrado. Retención de mensajes.	✓ ELEGIDA
HTTP REST polling	Sin broker adicional. Pero el dispositivo haría 6 req/min generando 60x más tráfico. Sin soporte para red inestable. Mayor consumo de batería.	✗ Descartada
WebSockets desde dispositivo	Bidireccional y eficiente. Pero los módulos ESP32 tienen soporte WebSocket limitado y la reconexión automática no es tan robusta como MQTT.	✗ Descartada
CoAP (Constrained Application Protocol)	Más ligero que MQTT para redes muy restringidas. Pero menor ecosistema, sin broker maduro disponible y complejidad de implementación en ESP32.	✗ Descartada

Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
• Overhead de protocolo mínimo (2 bytes vs ~200 bytes HTTP).	• Requiere broker MQTT adicional (Mosquitto) en la infraestructura.
• QoS Level 1 garantiza entrega al menos una vez sin pérdida de datos críticos.	• TLS añade overhead de handshake en reconexiones (mitigado con sesiones persistentes).
• Keep-alive integrado para detectar desconexiones sin polling activo.	• Autenticación de dispositivos requiere gestión de certificados de cliente.
• Last Will Testament: el broker notifica automáticamente si un dispositivo se desconecta.	• Debugging de mensajes MQTT más complejo que HTTP (sin curl directo).
• Reconexión automática configurable en el cliente del dispositivo.	•
• Retención de mensajes: el broker puede entregar el último mensaje a nuevos suscriptores.	•

Consecuencias

- Estructura de topics definida: smartcow/farms/{farmId}/animals/{animalId}/telemetry para datos en tiempo real. smartcow/devices/{deviceId}/config para configuración remota.
- QoS Level 1 (at least once) para telemetría. QoS Level 2 (exactly once) para eventos críticos de alerta del dispositivo.
- Cada dispositivo IoT se autentica con un certificado de cliente único firmado por la CA del sistema. Certificados con TTL de 2 años, renovables remotamente.
- El API Server usa MQTT.js con reconexión automática y buffer local de mensajes para tolerar reinicios del broker.

ADR-005 — React SPA con Vite en lugar de Next.js SSR

Estado	ACEPTADA
Fecha	22 de enero de 2026
Decisores	Tech Lead, Frontend Lead
Consultados	Equipo de Desarrollo
Informados	Product Owner, UX Designer

Contexto y Problema

El frontend de SmartCow Tracker es una aplicación de gestión de datos en tiempo real destinada a usuarios autenticados (no hay contenido público indexable). Toda la funcionalidad (mapas GPS en vivo, alertas, dashboards) requiere JavaScript habilitado y datos del usuario autenticado. Se debe elegir entre un framework de renderizado en servidor (SSR) como Next.js o una Single Page Application (SPA) pura como React con Vite.

Decisión

Se adopta React 18 con Vite 5 como framework de frontend, generando una SPA estática desplegable en CDN. Se descarta Next.js SSR por no aportar beneficios relevantes para este tipo de aplicación (dashboard autenticado sin SEO requerido) mientras añade complejidad de despliegue y un servidor Node.js adicional.

Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
React 18 + Vite 5 (SPA)	Build estático. CDN deployment. HMR instantáneo en desarrollo. Sin servidor adicional. Redux + React Query para estado. Carga inicial única, navegación instantánea después.	✓ ELEGIDA
Next.js 14 (App Router + SSR)	SSR/SSG integrado. Mejor para SEO y primera carga. Pero requiere servidor Node.js en producción, añade complejidad para una app detrás de login sin beneficio de SEO.	✗ Descartada
Vue 3 + Vite (SPA)	Alternativa válida, pero el ecosistema de mapas (React-Leaflet, React-Map-GL) y componentes UI (shadcn/ui) tiene mayor madurez y comunidad en React.	✗ Descartada
Angular 17	Framework opinado con todo incluido. Pero mayor curva de aprendizaje, bundle más pesado y menor velocidad de desarrollo para un equipo pequeño.	✗ Descartada

Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
• Build estático: deployment en Vercel/Cloudflare Pages sin servidor (coste \$0).	• Sin SEO en páginas internas (no es requisito, pero limita futuros públicos).
• HMR instantáneo con Vite: ciclos de desarrollo más rápidos.	• Tiempo de carga inicial mayor que SSR en conexiones lentas (mitigado con lazy loading y code splitting).
• Sin servidor SSR adicional que operar y escalar.	• Sin rendering en servidor: si JavaScript falla, la app no funciona.
• React-Leaflet para mapas: integración declarativa madura con Leaflet.js.	•
• Ecosistema React masivo: shadcn/ui, TanStack Query, Redux Toolkit.	•
• Carga inicial única; navegación posterior sin latencia de red (SPA).	•

Consecuencias

- El frontend se compila con vite build en CI/CD y se despliega en Vercel como archivos estáticos con caché agresivo (Cache-Control: max-age=31536000 para assets con hash).
- Code splitting automático con Vite: cada ruta del dashboard es un chunk lazy-loaded independiente para minimizar el bundle inicial.
- El tiempo de carga inicial objetivo es < 3 segundos en 10 Mbps (RNF-PE-001), logrado con lazy loading + compresión Brotli en CDN.
- El modo offline se implementa con Service Worker (Workbox) sobre la SPA compilada.

ADR-006 — Servicio de IA como contenedor Python independiente

Estado	ACEPTADA
Fecha	25 de enero de 2026
Decisores	Tech Lead, Data Scientist
Consultados	Equipo de Backend, Equipo de ML
Informados	Product Owner

| Contexto y Problema

SmartCow Tracker requiere un módulo de predicción de salud animal basado en análisis de series de tiempo de signos vitales. Las principales librerías de Machine Learning (scikit-learn, TensorFlow, pandas, NumPy) tienen su ecosistema primario en Python. El backend principal está en Node.js. Se debe decidir cómo integrar las capacidades de ML en el sistema sin comprometer la arquitectura existente.

| Decisión

Se despliega el módulo de ML como un microservicio Python independiente (FastAPI + scikit-learn) en su propio contenedor Docker. El API Server Node.js lo consume via HTTP interno en la misma red privada del cloud. Este es el único caso donde se justifica introducir un segundo lenguaje y un microservicio real en la arquitectura.

| Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
Microservicio Python + FastAPI	Contenedor Python independiente con FastAPI. Ecosistema ML nativo. Escalado independiente. Reemplazable sin afectar el API Server.	✓ ELEGIDA
Node.js + TensorFlow.js	Sin servicio adicional. Pero TensorFlow.js es significativamente menos maduro que su equivalente Python para entrenamiento. Menor comunidad de modelos preentrenados.	✗ Descartada
ML as a Service (AWS SageMaker)	Sin infraestructura que gestionar. Pero costo elevado, vendor lock-in fuerte y latencia de red a servicio externo en cada predicción.	✗ Descartada
Procesamiento batch nocturno (sin API)	Sin latencia en tiempo real. Pero las predicciones no están disponibles bajo demanda; solo se actualizan una vez al día, reduciendo su utilidad operacional.	✗ Descartada

| Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
Ecosistema ML completo: scikit-learn, pandas, NumPy, SHAP, joblib.	Latencia de red adicional en llamadas internas (~5-10ms en misma VPC, aceptable).
Modelo reemplazable sin redeployment del API Server principal.	Gestión de dos stacks tecnológicos (Node.js y Python) en CI/CD.
Escalado independiente: más réplicas del servicio IA solo cuando se necesitan.	Imagen Docker de Python con librerías ML es grande (~500MB).
FastAPI genera documentación OpenAPI automáticamente.	Consistencia de datos: el servicio necesita acceso a histórico de PostgreSQL o que el API Server le envíe los datos.
Circuit breaker en el cliente: si el servicio IA falla, el sistema continúa sin predicciones.	
Separación de responsabilidades: el equipo de ML puede iterar sin conocer Node.js.	

Consecuencias

- El API Server actúa como intermediario: cuando se solicita una predicción, recopila las últimas 72 horas de vitales de PostgreSQL y los envía al servicio Python en un único request. El servicio Python no tiene acceso directo a la base de datos.
- Se implementa un Circuit Breaker con 3 fallos consecutivos → open state → el endpoint de predicción devuelve un aviso 'Predicción temporalmente no disponible' sin lanzar error al usuario.
- El modelo se re-entrena semanalmente con un job automatizado. Los modelos se versionan con fechas (model_v20260528.joblib) y se almacenan en S3.
- La latencia máxima aceptable para el endpoint de predicción es 3 segundos (no es dato de tiempo real, es análisis bajo demanda).

ADR-007 — JWT RS256 con refresh token en cookie HttpOnly

Estado	ACEPTADA
Fecha	28 de enero de 2026
Decisores	Tech Lead, Security Lead
Consultados	Equipo de Desarrollo
Informados	Product Owner, Auditores de Seguridad

Contexto y Problema

El sistema necesita un mecanismo de autenticación stateless para la API REST que sea seguro contra los ataques más comunes (XSS, CSRF, token hijacking), soporte múltiples clientes (navegadores web, futura app móvil), permita control de roles granular y sea escalable horizontalmente sin sincronización de sesiones entre instancias.

Decisión

Se implementa autenticación basada en JWT (JSON Web Tokens) con firma RS256 (RSA + SHA-256 usando par de claves asimétricas). El access token (vida: 8 horas) se entrega en el body de la respuesta. El refresh token (vida: 30 días) se almacena exclusivamente en una cookie HttpOnly + Secure + SameSite=Strict para prevenir acceso desde JavaScript (protección XSS).

Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
JWT RS256 + cookie HttpOnly refresh	Access token en body (stateless). Refresh token en cookie HttpOnly (XSS-safe). Claves asimétricas: el servicio IA puede verificar tokens sin la clave privada.	✓ ELEGIDA
JWT HS256 + localStorage	Implementación más simple. Pero HS256 es simétrico (misma clave para firmar y verificar). localStorage es vulnerable a XSS.	✗ Descartada
Sesiones server-side (Redis)	Sin tokens en cliente. Pero requiere sincronización de sesiones entre instancias y añade latencia en cada request para verificar la sesión en Redis.	✗ Descartada
OAuth2 / Auth0	Delegación a proveedor externo. Sin gestión de identidad propia. Pero introduce dependencia de tercero, costo adicional y complejidad para usuarios ganaderos sin Google/GitHub.	✗ Descartada

Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
RS256 asimétrico: servicios secundarios (IA) verifican tokens con clave pública sin exponer la privada.	RS256 es más lento que HS256 para firma/verificación (mitigado: verificación es rápida, firma solo en login).
Cookie HttpOnly protege el refresh token contra robo via XSS.	Revocación de tokens requiere lista negra en Redis (invalidación ante logout o compromiso).
SameSite=Strict protege el refresh token contra CSRF.	Gestión del par de claves RSA requiere práctica de seguridad adicional (rotación, backup).
Stateless: sin base de datos para verificar cada request (O(1) verificación criptográfica).	
Payload del JWT incluye farmId y permisos: autorización sin query adicional a BD.	
Rotación de claves posible sin invalidar todos los tokens activos (usando kid).	

Consecuencias

- Las claves RSA se generan con openssl (4096 bits) y se almacenan en el secret manager del cloud. Se rotan anualmente o ante sospecha de compromiso.
- La lista negra de tokens revocados (logout, cambio de contraseña) se mantiene en Redis con TTL igual al tiempo restante de vida del token.
- El payload del JWT contiene: { sub: userId, farmId, role, permissions[], iat, exp, kid }. Los permisos son inmutables durante la vida del token (8h máximo).
- Ante cambio de rol de un usuario, el administrador puede invalidar sus tokens activos; el usuario deberá re-autenticarse.

ADR-008 — Prisma ORM sobre consultas SQL directas

Estado	ACEPTADA
Fecha	01 de febrero de 2026
Decisores	Tech Lead
Consultados	Equipo de Desarrollo
Informados	Product Owner

| Contexto y Problema

El backend Node.js necesita interactuar con PostgreSQL para todas las operaciones de datos. Se debe decidir si usar un ORM (Object-Relational Mapper) con tipado estático automático, un query builder de bajo nivel, o SQL puro con un driver de bajo nivel. La decisión impacta directamente en la velocidad de desarrollo, seguridad de tipos, riesgo de SQL injection y facilidad de mantenimiento.

| Decisión

Se adopta Prisma 5 como ORM principal para todas las operaciones de base de datos del backend. Prisma genera tipos TypeScript automáticamente desde el esquema, provee migraciones versionadas con prisma migrate y ofrece un cliente type-safe que previene errores de tipo en tiempo de compilación. Para consultas de alta performance que Prisma no optimiza bien (queries analíticas complejas), se permite el uso de prisma.\$queryRaw con prepared statements.

| Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
Prisma 5 ORM	Esquema declarativo. Tipos TypeScript auto-generados. Migraciones automáticas. Seguridad contra SQL injection integrada. DX excelente.	✓ ELEGIDA
Drizzle ORM	ORM más liviano y de bajo nivel. Muy similar a SQL. Mejor performance que Prisma para queries complejas. Pero menor madurez en migraciones y ecosistema más pequeño.	✗ Descartada
Knex.js (query builder)	Control total sobre el SQL generado. Sin tipado automático: requiere definir tipos manualmente. Mayor riesgo de errores y más código boilerplate.	✗ Descartada
node-postgres (pg) + SQL puro	Máximo control y performance. Pero sin protección automática contra SQL injection si se usan template strings, sin migraciones, sin tipos automáticos.	✗ Descartada

| Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
Tipos TypeScript generados automáticamente: errores detectados en compilación.	Overhead de abstracción: queries complejas generan SQL subóptimo a veces.
prisma migrate: historial de migraciones versionado en Git.	Prisma no soporta todas las características avanzadas de PostgreSQL (particionamiento, funciones específicas).
Prisma Studio: interfaz visual para explorar datos en desarrollo.	Migraciones automáticas requieren revisión cuidadosa en producción.
Protección automática contra SQL injection en todas las operaciones.	Bundle size mayor que alternativas de bajo nivel.
Relaciones y includes tipados: joins seguros sin SQL manual.	
Soporte completo para transacciones, upserts y operaciones batch.	

Consecuencias

- Para queries analíticas complejas (reportes, agregaciones sobre GpsHistory con millones de filas), se usa `prisma.$queryRaw` con prepared statements para control total del SQL.
- El esquema Prisma (`schema.prisma`) es la fuente de verdad del modelo de datos. Cualquier cambio en la BD se hace primero en el esquema y luego se genera la migración.
- Las migraciones se ejecutan automáticamente en el startup del servidor en entornos de staging. En producción, se ejecutan manualmente en ventana de mantenimiento tras revisión del SQL generado.

ADR-009 — Bull MQ para procesamiento asíncrono de trabajos

Estado	ACEPTADA
Fecha	05 de febrero de 2026
Decisores	Tech Lead
Consultados	Equipo de Desarrollo
Informados	DevOps, Product Owner

| Contexto y Problema

El sistema genera múltiples trabajos de procesamiento que no deben ejecutarse en el event loop principal del API Server: persistencia asíncrona de registros GPS (1,500 registros/min), evaluación de reglas de alerta, generación de reportes PDF/XLSX, inferencia del modelo de IA, reentrenamiento semanal del modelo y envío de notificaciones con reintentos. Ejecutar estas tareas en el proceso principal bloquearía el event loop y degradaría la latencia de la API REST.

| Decisión

Se adopta Bull MQ 5 como sistema de colas de trabajos asíncrono. Bull MQ usa Redis como backend de persistencia de jobs, lo que aprovecha la dependencia de Redis ya existente en el sistema sin añadir infraestructura nueva. Los jobs se procesan en el proceso Worker, que comparte código con el API Server pero se ejecuta como proceso separado.

| Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
Bull MQ 5 (Redis backend)	Cola de trabajos robusta sobre Redis existente. Reintentos automáticos con backoff exponencial. UI de monitoreo (Bull Board). Sin infraestructura adicional.	✓ ELEGIDA
RabbitMQ	Broker de mensajería maduro y muy flexible. Pero requiere servidor RabbitMQ adicional, protocolo AMQP diferente a MQTT, mayor complejidad operacional.	✗ Descartada
Node.js setInterval / setTimeout	Sin dependencias adicionales. Pero sin persistencia: si el proceso muere, los jobs pendientes se pierden. Sin reintentos automáticos ni monitoreo.	✗ Descartada
AWS SQS + Lambda	Totalmente gestionado. Pero introduce vendor lock-in, latencia de cold start en Lambda y costo variable difícil de predecir en alto volumen de mensajes.	✗ Descartada

| Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
------------	---------------

• Usa Redis ya existente: cero infraestructura adicional.	• Acoplamiento a Redis: si Redis falla, las colas también fallan.
• Persistencia de jobs: si el Worker muere, los jobs pendientes se recuperan al reiniciar.	• Bull MQ no es un message broker completo: no soporta pub/sub de alta disponibilidad.
• Reintentos automáticos con backoff exponencial configurable por tipo de job.	• Serialización: los argumentos de los jobs deben ser JSON-serializable.
• Prioridades de cola: jobs de alertas tienen mayor prioridad que generación de reportes.	•
• Bull Board: interfaz web de monitoreo de colas en desarrollo/staging.	•
• Delay jobs: programar ejecución futura (ej: reentrenamiento semanal del modelo IA).	•

Consecuencias

- Se definen las siguientes colas con sus prioridades: persist-gps (prioridad alta, concurrencia 10), evaluate-alerts (prioridad crítica, concurrencia 5), send-notifications (prioridad alta, reintentos 3), generate-reports (prioridad baja, concurrencia 2), ai-predict (prioridad media, concurrencia 3).
- Jobs fallidos después de los reintentos configurados se mueven a la cola Dead Letter Queue (DLQ) y generan una alerta al equipo de operaciones.
- Bull Board se expone en /admin/queues solo en entorno de desarrollo y staging, protegido con autenticación básica.

ADR-010 — Leaflet.js con OpenStreetMap sobre Google Maps

Estado	ACEPTADA
Fecha	08 de febrero de 2026
Decisores	Tech Lead, Frontend Lead
Consultados	UX Designer, Product Owner
Informados	Stakeholders, Equipo de Desarrollo

Contexto y Problema

El módulo de mapa GPS es el componente más visible y crítico de la interfaz de usuario de SmartCow Tracker. Debe mostrar hasta 247 marcadores actualizándose en tiempo real, soportar dibujo de geocercas poligonales, mostrar rutas históricas de animales y funcionar en zonas rurales con conectividad limitada. Se debe elegir entre las principales opciones de librería de mapas para web.

Decisión

Se adopta Leaflet.js 1.9 con tiles de OpenStreetMap como proveedor de mapas principal, integrado via React-Leaflet para uso declarativo en React. Para funcionalidades avanzadas de estilo (mapa satelital de alta calidad), se habilita opcionalmente Mapbox GL JS con tiles de Mapbox como capa alternativa configurable por el usuario.

Opciones Consideradas

Opción considerada	Descripción y evaluación	Resultado
Leaflet.js + OpenStreetMap	Librería open-source. Sin costo de API. Sin límites de uso. React-Leaflet para integración declarativa. Amplio ecosistema de plugins.	✓ ELEGIDA
Google Maps JavaScript API	Tiles de alta calidad y Street View. Pero costo: \$7 por 1,000 cargas de mapa. Con 100 usuarios activos diarios = ~\$210/mes. Vendor lock-in fuerte.	✗ Descartada
Mapbox GL JS	Excelente calidad visual y performance con WebGL. Pero plan gratuito limitado a 50,000 cargas/mes; costo posterior. Se adopta como capa opcional premium.	✗ Descartada
Deck.gl (Uber) + MapLibre	Renderizado WebGL de alto rendimiento para millones de puntos. Sobredimensionado para 247 marcadores. Curva de aprendizaje alta.	✗ Descartada

Argumentos a Favor y en Contra

✓ Ventajas	✗ Desventajas
------------	---------------

Costo cero: OpenStreetMap es open-source sin límites de API.	Calidad visual inferior a Google Maps o Mapbox en zonas rurales de Colombia.
React-Leaflet: integración declarativa y mantenida activamente.	Sin Street View ni capas de tráfico integradas.
Amplio ecosistema: leaflet-draw (geocercas), leaflet.markercluster, leaflet-routing.	Tiles satelitales de OSM tienen menor resolución que Google/Mapbox en algunas regiones.
Tiles descargables para caché offline (Service Worker + Cache API).	Mapa 3D no disponible nativamente (requiere Mapbox GL JS o Deck.gl).
Sin vendor lock-in: tiles intercambiables (OSM, Mapbox, Stamen, ESRI).	
Rendimiento adecuado para < 500 marcadores con optimización (memoización, virtualización).	

Consecuencias

- La capa de tiles por defecto es OpenStreetMap. El usuario puede cambiar a tiles de Mapbox satelital desde el selector de capas del mapa (requiere que el usuario tenga token de Mapbox configurado en ajustes de la finca).
- Los tiles del mapa para la zona geográfica de la finca se precargan en el Service Worker para disponibilidad offline (estrategia CacheFirst para tiles con TTL de 7 días).
- leaflet-draw se usa para el editor de geocercas: permite dibujar polígonos directamente sobre el mapa con snapping a vértices existentes.
- El rendimiento con 247 marcadores se garantiza con React.memo en AnimalMarker, virtualización del viewport y actualización selectiva (solo re-renderiza marcadores con posición cambiada en el último ciclo).

— Fin del Documento ADR v1.0.0 —

SmartCow Tracker | 10 decisiones arquitectónicas documentadas | Mayo 2026